framework. Their views typically include an `@model` directive that references the `ViewModel` class designed for the specific view:

```
@model OrderSummaryViewModel
```

You have already seen the `@model` directive in your Razor page files. It references the `PageModel` class that accompanies the Razor page. So the `PageModel` in Razor Pages is a view model in the MVC sense. Any public properties added to the `PageModel` will be accessible to the Razor page that references the `PageModel` via a special `Model` property within the page.

Listing 3.25 shows the `WelcomeModel`, the `PageModel` class for the Welcome page you created in the last chapter. This version has a public property named `SaleEnds`, which is the equivalent of the `ViewData` value you saw earlier.

Listing 3.25 Public properties in a `PageModel`

```
public class WelcomeModel : PageModel
{
    public DateTime SaleEnds { get; set; } = new DateTime(DateTime.Now.Year,
    ➥ 6, 30);
    public void OnGet()
    {
    }
}
```

Listing 3.26 shows the Welcome Razor page, which includes the `@model` directive referencing the `WelcomeModel` type and the `SaleEnds` value being accessed via the page's `Model` property.

Listing 3.26 `PageModel` properties being exposed via the page's `Model` property

```
@page
@model WebApplication1.Pages.WelcomeModel                    ❶
@{
}
<p>Sale ends at @Model.SaleEnds.ToString("h tt, MMMM dd")</p>    ❷
```